



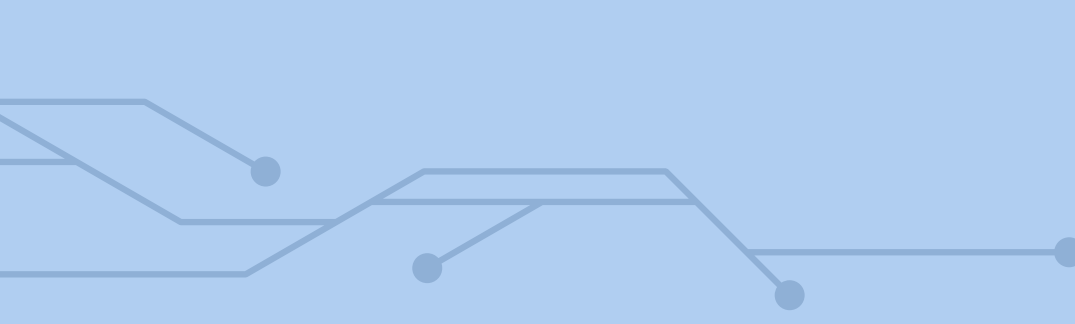
Cifra de **Vigenère**

Evelyn Caroline Morais Targino
Leonardo Marques de Paula Junior
Lucas Vasconcelos Pirani Carneiro
Vinicius Geraldo dos Santos Guerra

Cifra de Vigenère: Implementação e Criptoanálise

Este trabalho apresenta a implementação da cifra de Vigenère, incluindo os processos de cifragem e decifragem, bem como um ataque de recuperação de chave baseado em análise de frequência, conforme proposto na disciplina de Segurança Computacional.

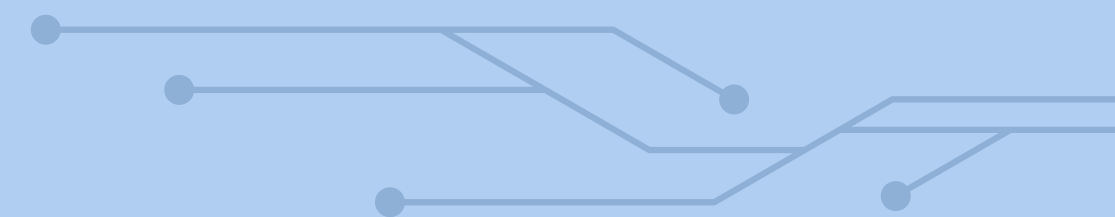


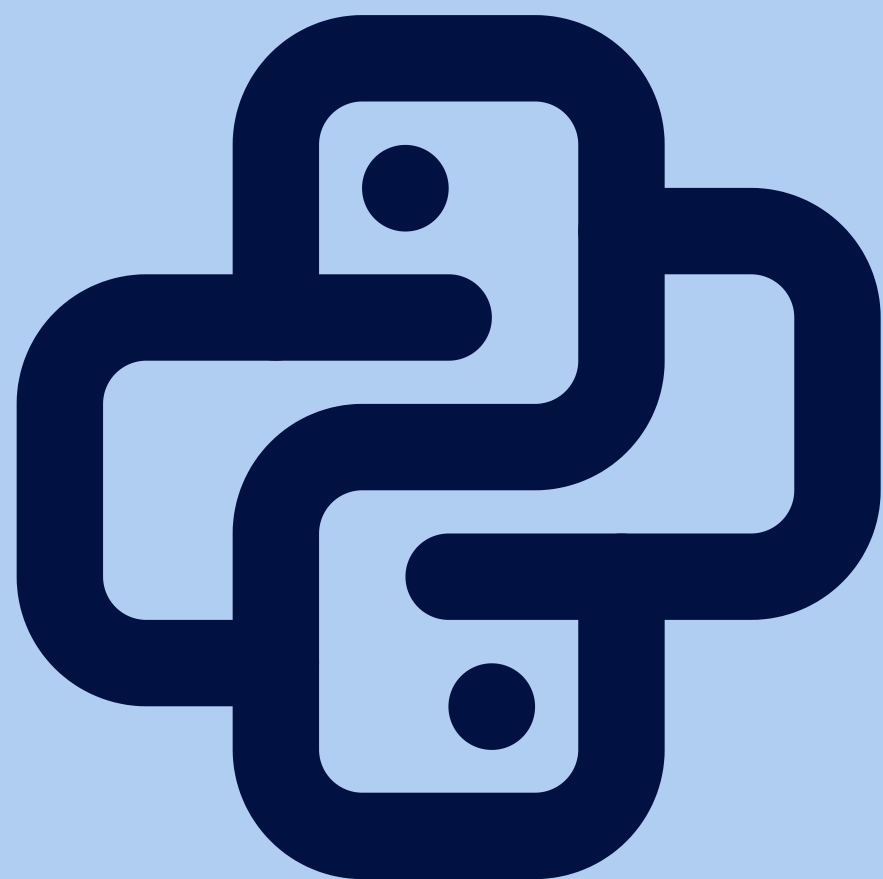


Parte I: Cifrador e Decifrador

A cifra de Vigenère é um método de substituição polialfabética que utiliza uma chave textual repetida ao longo da mensagem. Cada letra do texto é combinada com a letra correspondente da chave por meio de aritmética modular, permitindo gerar o texto cifrado. O processo inverso possibilita a recuperação da mensagem original.

```
1 def cifrador(texto, chave):
2     resultado=[]
3     indice=0
4     texto=texto.upper()
5     chave=chave.upper()
6     for x in texto:
7         if(x.isalpha()):
8             newletter=(ord(x)-65+ord(chave[indice])-65)%26
9             resultado.append(chr(newletter+65))
10            indice=indice+1
11            if (indice>=len(chave)):
12                indice=0
13        else:
14            resultado.append(x)
15    return "".join(resultado)
16
17 def decifrador(texto, chave):
18     resultado=[]
19     indice=0
20     texto=texto.upper()
21     chave=chave.upper()
22     for x in texto:
23         if(x.isalpha()):
24             newletter=(ord(x)-ord(chave[indice]))%26
25             resultado.append(chr(newletter+65))
26             indice=indice+1
27             if(indice>=len(chave)):
28                 indice=0
29        else:
30            resultado.append(x)
31    return "".join(resultado)
32
```





Implementação

A implementação foi realizada em Python, utilizando operações com códigos ASCII para converter letras em valores numéricos. O algoritmo trata apenas caracteres alfabéticos, mantendo símbolos e espaços inalterados. Observou-se como limitação o tratamento de caracteres acentuados, que não são corretamente processados pelo modelo adotado.

Parte II:

Ataque à cifra

A quebra da cifra foi realizada por meio da análise de frequência, explorando a repetição da chave. Inicialmente, utilizou-se o Índice de Coincidência para estimar o tamanho da chave. Em seguida, o texto foi dividido em subconjuntos, permitindo aplicar técnicas semelhantes à cifra de César para recuperar cada caractere da chave.

```
breaker.py > ...
1  def ic(texto):
2      texto = texto.upper()
3      tamanho=len(texto)
4      frequencia = {}
5      for x in texto:
6          frequencia[x] = frequencia.get(x, 0) + 1
7      num = sum (f*(f-1) for f in frequencia.values())
8      den = tamanho*(tamanho-1)
9      return num/den
10
11 def tamanho (texto , max_len=20):
12     texto=texto.upper()
13     melhor_tam = 1
14     maior_ic = 0
15     for tam in range (1,max_len + 1):
16         ics = []
17         for x in range (tam):
18             pedaco=texto[x::tam]
19             ics.append(ic(pedaco))
20             media_ic=sum(ics)/len(ics)
21
22             if media_ic>maior_ic:
23                 maior_ic=media_ic
24                 melhor_tam=tam
25     return melhor_tam
26
27 pt=[0.1463, 0.0104, 0.0388, 0.0499, 0.1257, 0.0102, 0.0130, 0.0128, 0.0618,
28     0.0040, 0.0002, 0.0278, 0.0474, 0.0505, 0.1073, 0.0252, 0.0120, 0.0653,
29     0.0781, 0.0434, 0.0463, 0.0167, 0.0001, 0.0021, 0.0001, 0.0047]
30
31 en=[0.0817, 0.0149, 0.0278, 0.0425, 0.1270, 0.0223, 0.0202, 0.0609, 0.0697,
32     0.0015, 0.0077, 0.0403, 0.0241, 0.0675, 0.0751, 0.0193, 0.0010, 0.0599,
33     0.0633, 0.0906, 0.0276, 0.0098, 0.0236, 0.0015, 0.0197, 0.0007]
34
35 def cesar(texto, n):
36     resultado = ""
37     for i in range(len(texto)):
```

Ativar
Acesse C

Resultados e Conclusão

Os resultados demonstraram que a cifra de Vigenère é vulnerável quando utiliza chaves curtas ou repetitivas. O método de análise estatística foi capaz de recuperar a chave e decifrar corretamente as mensagens. Conclui-se que a segurança da cifra depende diretamente do tamanho e da aleatoriedade da chave utilizada.

